AllergyLens — 1-Hour Computer Vision App Spec

Concept

AllergyLens is a computer vision app that lets users scan a food ingredient label and instantly see whether the product contains allergens or dietary concerns.

The first version focuses on:

- Taking/uploading a photo of a food label
- Extracting text with OCR
- Parsing the ingredients
- Detecting common allergens
- Showing a clear safety/risk report

MVP Goal

By the end of the first 1-hour session, the app should let a user:

1. Upload or take a photo of an ingredient label
2. Extract readable text from the image
3. Detect common allergens
4. Display a simple result screen

Target Users

People who want to quickly check food products for:

- Peanut allergy
- Tree nut allergy
- Dairy allergy
- Egg allergy
- Soy allergy
- Wheat/gluten
- Shellfish
- Fish
- Sesame
- GERD/LPR triggers
- Histamine concerns

Core User Flow

1. User opens app
2. User taps “Scan Label”
3. User uploads/takes photo
4. App performs OCR
5. App extracts ingredients
6. App compares ingredients against allergen database
7. App shows result:
8. Safe
9. Warning
10. Contains allergen
11. Unknown / needs review

Tech Stack

Recommended stack:

- Next.js
- TypeScript
- Tailwind CSS
- Tesseract.js for OCR, or OpenAI/Claude vision API
- Local JSON allergen database
- Vercel for deployment

Folder Structure

```
/allergy-lens
  /app
    page.tsx
    scan/page.tsx
    results/page.tsx

  /components
    CameraUpload.tsx
    ScanButton.tsx
    ResultCard.tsx
    AllergenBadge.tsx
    IngredientList.tsx
    RiskScore.tsx
```

```
  /lib
    ocr.ts
    parser.ts
    allergenEngine.ts
    riskScoring.ts

  /data
    allergens.json
    ingredientAliases.json

  /types
    scan.ts
    allergens.ts

  README.md
  SPEC.md
```

Branch Strategy

Use this structure:

```
main
develop

feature/app-shell
feature/camera-upload
feature/ocr
feature/parser
feature/allergen-engine
feature/results-ui
feature/allergen-database
feature/history
feature/polish
```

For a 2-person team, keep it simple during the 1-hour sprint:

```
main
develop

brandon/ui-camera-results
friend/ocr-parser-engine
```

Work Split

Brandon

Owns the user experience and visual side.

Branch:

```
brandon/ui-camera-results
```

Responsibilities:

- App layout
- Home screen
- Upload/camera UI
- Results screen
- Allergen badges
- Ingredient list display
- Loading states
- Error states
- Final polish

Files Brandon should mostly touch:

```
/app/page.tsx
/app/scan/page.tsx
/app/results/page.tsx
/components/*
```

Friend

Owns the logic and AI/computer vision side.

Branch:

```
friend/ocr-parser-engine
```

Responsibilities:

- OCR extraction

- Ingredient text cleanup
- Ingredient parsing
- Allergen matching
- Risk scoring
- Allergen database
- Types/interfaces

Files friend should mostly touch:

```
/lib/ocr.ts
/lib/parser.ts
/lib/allergenEngine.ts
/lib/riskScoring.ts
/data/allergens.json
/data/ingredientAliases.json
/types/*
```

MVP Features

1. Upload Label Image

User can upload an image of an ingredient label.

Acceptance criteria:

- Image preview appears
- User can remove and replace image
- App shows loading state when analyzing

---

2. OCR Extraction

App extracts text from the uploaded image.

Acceptance criteria:

- Extracted text is visible in developer/debug mode
- App can detect an ingredients section
- App handles bad/unclear images gracefully

---

3. Ingredient Parser

Convert OCR text into a clean ingredient list.

Example input:

```
Ingredients: enriched wheat flour, sugar, soybean oil, whey, soy lecithin,
natural flavors.
```

Example output:

```
[
  "enriched wheat flour",
  "sugar",
  "soybean oil",
  "whey",
  "soy lecithin",
  "natural flavors"
]
```

---

4. Allergen Engine

Compare ingredients against known allergens and aliases.

Example:

```
{
  "dairy": ["milk", "whey", "casein", "cream", "butter"],
  "soy": ["soy", "soybean", "soy lecithin"],
  "wheat": ["wheat", "flour", "gluten"],
  "egg": ["egg", "egg whites", "albumin"],
  "peanut": ["peanut", "groundnut"],
  "tree_nut": ["almond", "walnut", "cashew", "pecan"],
  "sesame": ["sesame", "tahini"],
  "fish": ["fish", "anchovy", "salmon", "tuna"],
  "shellfish": ["shrimp", "crab", "lobster"]
}
```

---

5. Results Screen

Show a simple decision.

Result types:

```
Safe
Warning
Contains Allergen
Unable to Determine
```

Example:

```
Contains Allergens

Detected:
- Wheat
- Dairy
- Soy

Ingredients flagged:
- enriched wheat flour
- whey
- soy lecithin

Confidence:
Medium
```

Risk Levels

Use 3 levels:

```
High Risk
Medium Risk
Low Risk
```

High Risk:

- Direct allergen found
- Example: milk, peanut, wheat, egg

Medium Risk:

- Alias or ambiguous ingredient found
- Example: natural flavors, lecithin, starch

Low Risk:

- No known allergens found

Data Models

ScanResult

```
type ScanResult = {
  rawText: string;
  ingredients: string[];
  detectedAllergens: DetectedAllergen[];
  warnings: string[];
  riskLevel: "low" | "medium" | "high" | "unknown";
  confidence: "low" | "medium" | "high";
};
```

DetectedAllergen

```
type DetectedAllergen = {
  allergen: string;
  matchedIngredient: string;
  severity: "low" | "medium" | "high";
  reason: string;
};
```

Nice-To-Have Features

Only add these after MVP works:

- Scan history
- User allergy profile
- GERD/LPR mode
- Histamine mode
- Low FODMAP mode
- Barcode scanner
- Product database lookup
- AI ingredient explanations
- Export/share report
- “Can I eat this?” AI summary

1-Hour Build Plan

Minute 0–5: Setup

- Create repo
- Create branches
- Agree on MVP
- Create mock scan result

Minute 5–20: Parallel Build

Brandon:

- Build home screen
- Build upload screen
- Build mock result UI

Friend:

- Build OCR function
- Build allergen database
- Build parser function

Minute 20–40: Connect Pieces

- Connect uploaded image to OCR
- Connect OCR text to parser
- Connect parser to allergen engine
- Send result to UI

Minute 40–50: Polish

- Add loading state
- Add error state
- Improve mobile layout
- Add sample test image/manual input fallback

Minute 50–60: Merge + Demo

- Merge into develop
- Fix conflicts
- Push to main if working
- Deploy to Vercel

Git Workflow

Start:

```
git checkout main
git pull
git checkout -b develop
git push -u origin develop
```

Brandon:

```
git checkout develop
git checkout -b brandon/ui-camera-results
```

Friend:

```
git checkout develop
git checkout -b friend/ocr-parser-engine
```

Every 15 minutes:

```
git add .
git commit -m "progress update"
git pull origin develop
git push
```

End of session:

```
git checkout develop
git pull
git merge brandon/ui-camera-results
git merge friend/ocr-parser-engine
git push origin develop
```

Then:

```
git checkout main
git merge develop
git push origin main
```

AI Agent Instructions

Use this prompt for Brandon's AI session:

```
You are helping build the UI for AllergyLens.

Only edit files in:
- /app
- /components
- styling files

Do not edit:
- /lib
- /data
- /types
- package.json

Build a clean, mobile-first interface for scanning food labels and displaying
allergen results.
```

Use this prompt for friend's AI session:

```
You are helping build the OCR and allergen detection logic for AllergyLens.

Only edit files in:
- /lib
- /data
- /types

Do not edit:
- /app
- /components
- styling files

Build functions for OCR text extraction, ingredient parsing, allergen matching,
and risk scoring.
```

Definition of Done

The MVP is done when:

- A user can upload a food label image
- OCR extracts text or fallback manual text works

- Ingredients are parsed
- Allergens are detected
- A result screen clearly shows safe/warning/unsafe
- The app runs locally without errors

Future Version Name Ideas

- AllergyLens
- LabelGuard
- SafeBite
- FoodRadar
- IngredientIQ
- BiteCheck
- ScanSafe
- LabelSense

Recommended name:

```
AllergyLens
```

Tagline:

```
Scan the label. Know the risk.
```